

Engenharia de Software – PUC Campinas

Estrutura e Recuperação de Dados I

Prof. Dr. Paulo Cesar de Alvarenga Lucci

Exercício de Pilha:

Validação de expressão aritmética (parênteses balanceados)

Validação de expressão aritmética (parênteses balanceados)

○ OBJETIVO:

- Usar uma pilha para verificar se os **delimitadores** de uma expressão matemática estão balanceados.
 - **Delimitadores** são: {}, [] e ().

○ EXERCÍCIO:

- Escreva um programa que leia uma expressão matemática contendo parênteses, colchetes e chaves e use uma **pilha** para verificar se esses delimitadores estão balanceados.
- Se a expressão estiver corretamente balanceada, o programa deve exibir:
 - **"Expressão válida"**.
- Caso contrário, deve exibir:
 - **"Expressão inválida"**.

○ EXEMPLO:

- Entrada: $\{[(2+3)*5] + (4/2)\}$ Saída: **return true;** (**Expressão válida**)
- Entrada: $\{[(2+3)*5\} + (4/2)\}$ Saída: **return false;** (**Expressão inválida**)

Validação de expressão aritmética (parênteses balanceados)

○ REGRAS PARA QUE A EXPRESSÃO ESTEJA BALANCEADA:

- Para cada **delimitador de abertura**, deve existir um **delimitador de fechamento** correspondente.
- Por exemplo, na expressão: $\{[(2+3)*5] + (4/2)\}$
- O primeiro delimitador é uma chave **{**, então esperamos que a expressão contenha uma chave **}** para fechá-la.
- Dentro dessas chaves, temos colchetes **[]**, que também precisam ser fechados corretamente.
- Finalmente, há parênteses **()**, que também precisam ser fechados corretamente.

Validação de expressão aritmética (parênteses balanceados)

- PROBLEMA:

- A quantidade de delimitadores de **abertura** precisa ser igual à quantidade de delimitadores de **fechamento**.
- Além disso... Não podemos **abrir** e **fechar** delimitadores de qualquer jeito.
 - Os delimitadores devem ser **fechados** na ordem inversa em que foram **abertos**, ou seja:
 - O último delimitador aberto deve ser o primeiro a ser fechado.

Validação de expressão aritmética (parênteses balanceados)

- COMO A PILHA É UTILIZADA PASSO A PASSO:
 - Percorrer a expressão caractere por caractere.
 - Para cada símbolo encontrado:
 - Se o símbolo for de abertura: { ou [ou (
 - Empilhe o símbolo na pilha.
 - Se o símbolo for de fechamento: } ou] ou)
 - Verifique!!!

Validação de expressão aritmética (parênteses balanceados)

○ VERIFICAÇÕES SE FOR DELIMITADOR DE FECHAMENTO:

1. Se a pilha estiver vazia:

→ Não houve nenhum delimitador de **abertura** correspondente!

→ Portanto: **return false**; (**Expressão inválida**).

2. Se a pilha **não** estiver vazia:

→ Retire (**desempilhe**) o **topo**.

→ **Verifique** se o topo removido é o **tipo correto de abertura**:

○ { deve casar com }

○ [deve casar com]

○ (deve casar com)

→ Se não corresponder: **return false**; (**Expressão inválida**).

→ Caso contrário, **continue!!!**

Validação de expressão aritmética (parênteses balanceados)

- APÓS PERCORRER TODA A EXPRESSÃO

- Se o final da expressão for atingido:

- Se a pilha estiver vazia:

- Todos os delimitadores que foram abertos foram fechados corretamente!!!

- `return true;` (**Expressão válida**).

- Se ainda restar algo na pilha:

- Há delimitadores **abertos** que nunca foram **fechados**.

- `return false;` (**Expressão inválida**).

Assinatura da função: `bool verificarExpressao(char exp[])`